

**UNIVERSIDAD DE LA
SIERRA**



Universidad de la Sierra
"Una puerta abierta al cambio y al desarrollo"

ANALIZADOR LÉXICO

ALUMNA

ARVAYO COTA CRISTAL ALEJANDRA

MATERIA

COMPILADORES

CARRERA

INGENIERIA EN SISTEMAS COMPUTACIONALES

PROFESOR

ING. OMAR BUSTAMANTE CISNEROS

05 DE JUNIO DE 2026

FUNCIONAMIENTO DEL LEXER

El analizador léxico (**lexer**) es la primera fase de un compilador. Su función principal es leer el código fuente carácter por carácter, agrupar esos caracteres en unidades con significado llamadas **tokens** y clasificarlos según su tipo.

En este proyecto se desarrolló un lexer en Java capaz de leer un archivo de código fuente, identificar componentes léxicos y generar una lista estructurada de tokens con su tipo, lexema, línea y columna.

ESTRUCTURA DEL PROGRAMA

El proyecto está organizado en cuatro clases:

- **TokenType.java:** Enum que define todos los tipos de tokens que el lexer puede reconocer.
- **Token.java:** clase que representa un token individual almacenando su tipo, lexema, línea y columna.
- **Lexer.java:** clase principal que contiene la lógica de análisis léxico. Lee el código fuente y genera la lista de tokens.
- **Main.java:** punto de entrada del programa. Lee el archivo de código fuente, invoca al lexer e imprime los resultados.

TIPOS DE TOKENS IMPLEMENTADOS

El lexer reconoce los siguientes tipos de tokens:

- **Palabras reservadas:** int, float, if, else, while, for, return, true, false.
- **Identificadores:** Nombres de variables y funciones definidos por el usuario. Ejemplo: x, contador, resultado
- **Literales numéricos:**
 - **Enteros:** números sin punto decimal. Ejemplo: 10, 25, 100
 - **Decimales:** números con punto decimal. Ejemplo: 3.14, 0.5
- **Literales de cadena:** texto entre comillas dobles. Ejemplo: "Hola Mundo", "Juan"
- **Operadores aritméticos:** suma (+), resta (-), multiplicación (*), división (/)

- **Operadores de comparación:**
 - Igual (==), Diferente (!=), Asignación (=)
 - Menor que (<), Mayor que(>)
 - Menor o igual (<=), Mayor o igual (>=)
- **Delimitadores:**
 - Punto y coma (;), coma (,)
 - Parentesis (), Llaves { }, Corchetes []
- **Especiales:**
 - **EOF:** indica el fin del archivo
 - **ERROR:** carácter no reconocido por el lexer

EJEMPLOS DE ENTRADA Y SALIDA

ARCHIVO DE ENTRADA (programa.txt):

```
int x = 10;

float resultado = 3.14;

if (x > 5) {
    return resultado;
}
```

SALIDA GENERADA:

```
Token(INT      , "int"      , linea=1, columna=0)
Token(IDENTIFIER  , "x"        , linea=1, columna=4)
Token(ASSIGN     , "="        , linea=1, columna=6)
Token(INT_LITERAL , "10"       , linea=1, columna=8)
Token(SEMICOLON  , ";"        , linea=1, columna=10)
Token(FLOAT      , "float"    , linea=2, columna=0)
```

Token(IDENTIFIER , "resultado" , linea=2, columna=6)

Token(ASSIGN , "=" , linea=2, columna=16)

Token(FLOAT_LITERAL , "3.14" , linea=2, columna=18)

Token(SEMICOLON , ";" , linea=2, columna=22)

Token(IF , "if" , linea=3, columna=0)

Token(LPAREN , "(" , linea=3, columna=3)

Token(IDENTIFIER , "x" , linea=3, columna=4)

Token(GREATER_THAN , ">" , linea=3, columna=6)

Token(INT_LITERAL , "5" , linea=3, columna=8)

Token(RPAREN , ")" , linea=3, columna=9)

Token(LBRACE , "{" , linea=3, columna=11)

Token(RETURN , "return" , linea=4, columna=4)

Token(IDENTIFIER , "resultado" , linea=4, columna=11)

Token(SEMICOLON , ";" , linea=4, columna=20)

Token(RBRACE , "}" , linea=5, columna=0)

Token(EOF , "EOF" , linea=5, columna=1)

DIFICULTADES ENCONTRADAS

Durante el desarrollo del proyecto, se me dificultó lo siguiente:

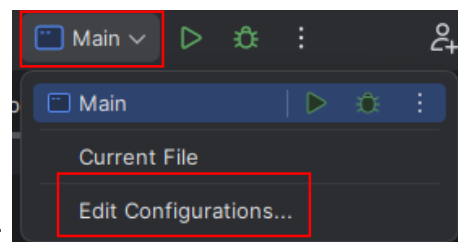
- Distinguir keyboards de identificadores: ambos comienzan con una letra, por lo que fue necesario primero leer la palabra completa y después ir a consultarla con el HashMap de palabras reservadas.
- Detectar operadores dobles: operadores como ==, !=, <= y >= requieren mirar el siguiente carácter sin consumirlo, lo cual se resolvió con el método peekNext().
- Manejo de strings sin cerrar: si el archivo contiene una cadena de texto sin comilla de cierre, el lexer la registra como token de tipo ERROR en lugar de crashear el programa.

INSTRUCCIONES DE USO

Requisitos: JDK 21 (de preferencia), IntelliJ Idea Community Edition (recomendado)

Pasos para ejecutarlo:

1. Abrir IntelliJ y seleccionar “open”
2. Seleccionar la carpeta del proyecto y abrirla
3. Colocar el archivo (programa.txt) a analizar en la raíz del proyecto



4. Ir a run, luego a Edit Configurations:
5. En “program arguments” escribir: programa.txt
6. Presionar el botón “run” o Shift + F10
7. La salida de tokens aparecerá en la consola.

En conclusión...

Se detectaron errores rápidamente gracias a la ayuda y apoyo de la inteligencia artificial, el analizador léxico me permitió comprender como un compilador transforma texto plano en estructuras que posteriormente se utilizarían como parser.

Se lograron implementar exitosamente los requerimientos solicitados: lectura de archivo, generación y clasificación de tokens, manejo de errores y organización de código en clases separadas.

La combinación de técnicas manuales y expresiones regulares dieron como resultado un lexer legible, funcional y fácil de extender con nuevos tipos de tokens en el futuro.

Me gusto realizar la actividad, me pareció muy interesante el saber y conocer cómo es que funciona el analizador léxico.